...

Giovanni Giuseppe Iacuzzo[1,*]

[1]Dipartimento di Ingegneria e Architettura, Università degli Studi di Enna "Kore", Enna, Italia

[*]Autore corrispondente: giovanni.iacuzzo@unikore.it

Sommario

...

Parole chiave: ...

1 Introduzione

I problemi di ottimizzazione continua su larga scala, con centinaia o migliaia di variabili decisionali, ricorrono in molti ambiti applicativi, dalla progettazione ingegneristica all'apprendimento automatico [1]. Spesso sono anche multimodali, una caratteristica che li avvicina ai problemi reali [2]. Gli algoritmi basati su popolazione, tra cui la Particle Swarm Optimization (PSO) [3], soffrono della cosiddetta «maledizione della dimensionalità»: le loro prestazioni peggiorano rapidamente al crescere del numero di variabili [4–6]. A questa difficoltà se ne affianca una distinta, il costo della singola valutazione della funzione obiettivo, che in molte applicazioni ingegneristiche è elevato [7]. I due aspetti sono indipendenti ma si presentano spesso insieme, e motivano in modo diverso il ricorso al calcolo parallelo.

Per ridurre i tempi di calcolo sono state proposte numerose implementazioni parallele e distribuite del PSO, su cluster [7, 8] e su GPU [9], sistematizzate nella rassegna di Lalwani et al. [6]. Esse si inseriscono nel quadro più ampio degli algoritmi evolutivi paralleli [10, 11], dove è essenziale distinguere il modello algoritmico dalla sua implementazione parallela [10]. Una valutazione parallela e sincrona delle particelle da parte di un processo master (schema master–slave) lascia invariata la traiettoria di ricerca dell'algoritmo seriale e ne riduce soltanto il tempo di esecuzione. Gli schemi asincroni, a isole (multi-swarm) o cellulari modificano invece le informazioni disponibili alle particelle e quindi la dinamica della ricerca. Anche la sincronizzazione incide sull'efficienza: quando i tempi di valutazione sono disomogenei, la barriera a ogni iterazione costringe ad attendere la valutazione più lenta e limita lo speedup [7], limite che le versioni asincrone mirano a ridurre [8].

In questo lavoro studiamo la parallelizzazione della formulazione continua del PSO proposta da Orlando and Ricciardello [12], che indichiamo con CPSO. In tale modello il moto di ogni particella non è descritto da una ricorrenza in tempo discreto, ma da una funzione continua del tempo, soluzione di un problema di Cauchy per un'equazione differenziale ordinaria del secondo ordine con condizioni iniziali casuali; gli autori ne ricavano la soluzione analitica e analizzano la convergenza al variare dei parametri. L'aggettivo «continuo» si riferisce quindi al tempo e non alle variabili di decisione, dato che anche il PSO standard opera su spazi continui. Precisiamo inoltre che l'acronimo CPSO è usato in letteratura anche per il Cooperative PSO di van den Bergh and Engelbrecht [4] e per un diverso modello a tempo continuo introdotto da Emara and Abdel Fattah [13]; in questo articolo designa esclusivamente il modello di Orlando and Ricciardello [12].

La descrizione del PSO in tempo continuo ha precedenti sul piano teorico: Clerc and Kennedy [14] analizzano la traiettoria

di una particella anche in tempo continuo, mentre Huang et al. [15] formulano le dinamiche come sistema di equazioni differenziali stocastiche, stabilendo garanzie di convergenza globale sotto opportune ipotesi e proponendo un'implementazione parallelizzabile, verificata su un problema di apprendimento automatico ad alta dimensione. Si tratta tuttavia di modelli diversi da CPSO. Per quanto a nostra conoscenza, il modello di Orlando and Ricciardello [12] non è stato finora indagato sistematicamente né in versione parallela né su problemi di grande dimensione.

Parallelizzare CPSO solleva una questione che non si pone per il PSO discreto. Poiché la posizione di ogni particella è nota in forma analitica come funzione del tempo, la nozione di «iterazione» non è intrinseca al modello: gli istanti in cui la funzione obiettivo viene valutata e le informazioni sociali (migliori posizioni personali e di vicinato) vengono aggiornate sono una scelta progettuale, e da essa dipende il significato stesso di esecuzione sincrona o asincrona.

Per inquadrare queste scelte proponiamo una tassonomia a quattro assi: (i) il *livello di parallelismo*, ossia l'entità su cui si distribuisce il calcolo: la singola valutazione (master–slave), la particella (grana fine o cellulare), lo sciame (isole o multi-swarm) oppure le variabili del problema, nella decomposizione cooperativa di Potter and De Jong [16], resa efficace su larga scala dal raggruppamento differenziale [5] e già applicata al PSO [2, 4]; (ii) la *topologia logica* di scambio delle informazioni (globale a stella, ad anello, a griglia), che è una proprietà dell'algoritmo anche in versione seriale [17] e diventa un vincolo di comunicazione solo quando viene mappata sull'hardware; (iii) lo *schema di sincronizzazione* (sincrono, asincrono, con scambi periodici); (iv) l'*architettura di esecuzione* (memoria condivisa, memoria distribuita, GPU). Le categorie del primo asse derivano in buona parte dalla classificazione di Cantú-Paz [11], sviluppata per gli algoritmi genetici (master–slave, demi multipli, grana fine e gerarchica), e sono state adattate al PSO in lavori successivi [6]; la distinzione tra scelte che preservano l'algoritmo e scelte che ne alterano la dinamica, e la sua lettura nel caso di un modello a tempo continuo, è invece un contributo di questo lavoro.

L'obiettivo dello studio è valutare empiricamente come tali scelte influiscano sulla qualità delle soluzioni e sull'efficienza di CPSO su problemi di grande dimensione. In particolare, ci poniamo tre domande di ricerca:

- **RQ1.** Come si possono definire e implementare le principali architetture parallele per CPSO, e quali schemi di sincronizzazione risultano ben definiti in un modello a tempo continuo?
- **RQ2.** Come variano qualità della soluzione, numero di valutazioni necessarie e speedup al variare di livello di parallelismo, topologia e sincronizzazione?
- **RQ3.** In quali condizioni CPSO parallelo ottiene risultati migliori o peggiori di varianti standard di PSO, a parità di budget computazionale e di hardware?

La valutazione sperimentale utilizza tre suite: bbob della piattaforma COCO [18], con 24 funzioni senza rumore in dimensione da 2 a 40; la sua estensione bbob-largescale, con dimensioni da 20 a 640 [19]; e le 15 funzioni a 1000 variabili della competizione CEC'2013 sull'ottimizzazione su larga scala [1]. COCO misura il tempo come numero di chiamate alla funzione obiettivo e la prestazione come tempo necessario per raggiungere valori obiettivo prefissati [18]: una metrica di questo tipo non cattura lo speedup di una versione parallela, che dipende dal tempo di esecuzione reale. Per questo valuteremo separatamente (a) la qualità della soluzione e il numero di valutazioni necessarie a raggiungere i valori obiettivo, secondo il protocollo COCO, e (b) tempo reale, speedup, efficienza parallela e scalabilità, a parità di budget di valutazioni e di hardware. Poiché le funzioni di test sono poco costose rispetto alle applicazioni reali, il rapporto tra calcolo e comunicazione sarebbe sfavorevole alla parallelizzazione; per esplorare anche il regime di valutazioni costose [7] introdurremo un costo di valutazione controllato. Come riferimento useremo il PSO standard [20] e, ove opportuno, varianti per larga scala [2, 21]; la significatività statistica sarà valutata con test non parametrici.

In sintesi, il lavoro offre i seguenti contributi:

- una tassonomia delle architetture parallele per CPSO che unifica livello di parallelismo, topologia, sincronizzazione e architettura di esecuzione, distingue le scelte che preservano l'algoritmo da quelle che ne alterano la dinamica e definisce la sincronizzazione in un modello a tempo continuo (Sezione **??**);
- l'implementazione di CPSO in configurazioni rappresentative (master–slave, isole, cellulare; topologie globale, ad anello e a griglia; esecuzione sincrona e asincrona);
- un protocollo sperimentale che separa la valutazione della qualità, secondo la metodologia COCO, da quella dell'efficienza parallela, applicato a bbob, bbob-largescale e CEC'2013;
- un confronto con varianti standard di PSO a parità di condizioni, volto a individuare quando la formulazione continua risulta vantaggiosa o svantaggiosa.

Il resto dell'articolo è organizzato come segue. La Sezione **??** passa in rassegna i lavori correlati su PSO, algoritmi evolutivi paralleli, ottimizzazione su larga scala e modelli a tempo continuo. La Sezione **??** presenta il modello matematico di CPSO e la sua implementazione. La Sezione **??** introduce la tassonomia e descrive le architetture parallele considerate. La Sezione **??** illustra il protocollo sperimentale, con funzioni di test, metriche e hardware. La Sezione **??** riporta i risultati su qualità, convergenza e scalabilità. La Sezione **??** conclude con una sintesi e gli sviluppi futuri.

Disponibilità di dati e codice

Riferimenti bibliografici

[1] Xiaodong Li, Ke Tang, Mohammad N. Omidvar, Zhenyu Yang, and Kai Qin. Benchmark functions for the CEC'2013 special session and competition on large-scale global optimization. Technical report, Evolutionary Computation and Machine Learning Group, RMIT University, Melbourne, Australia, 2013.

[2] Xiaodong Li and Xin Yao. Cooperatively coevolving particle swarms for large scale optimization. *IEEE Transactions on Evolutionary Computation*, 16(2):210–224, 2012. doi: 10.1109/TEVC.2011.2112662.

[3] James Kennedy and Russell Eberhart. Particle swarm optimization. In *Proceedings of ICNN'95 – International Conference on Neural Networks*, volume 4, pages 1942–1948, Perth, Australia, 1995. IEEE. doi: 10.1109/ICNN.1995.488968.

[4] Frans van den Bergh and Andries P. Engelbrecht. A cooperative approach to particle swarm optimization. *IEEE Transactions on Evolutionary Computation*, 8(3): 225–239, 2004. doi: 10.1109/TEVC.2004.826069.

[5] Mohammad Nabi Omidvar, Xiaodong Li, Yi Mei, and Xin Yao. Cooperative co-evolution with differential grouping for large scale optimization. *IEEE Transactions on Evolutionary Computation*, 18(3):378–393, 2014. doi: 10.1109/TEVC.2013.2281543.

[6] Soniya Lalwani, Harish Sharma, Suresh C. Satapathy, Kusum Deep, and Jagdish C. Bansal. A survey on parallel particle swarm optimization algorithms. *Arabian Journal for Science and Engineering*, 44(4):2899–2923, 2019. doi: 10.1007/s13369-018-03713-6.

[7] Jaco F. Schutte, Jeffrey A. Reinbolt, Benjamin J. Fregly, Raphael T. Haftka, and Alan D. George. Parallel global optimization with the particle swarm algorithm. *International Journal for Numerical Methods in Engineering*, 61 (13):2296–2315, 2004. doi: 10.1002/nme.1149.

[8] Byung-Il Koh, Alan D. George, Raphael T. Haftka, and Benjamin J. Fregly. Parallel asynchronous particle swarm optimization. *International Journal for Numerical Methods in Engineering*, 67:578–595, 2006. doi: 10.1002/nme.1646.

[9] Luca Mussi, Fabio Daolio, and Stefano Cagnoni. Evaluation of parallel particle swarm optimization algorithms within the CUDA architecture. *Information Sciences*, 181 (20):4642–4657, 2011. doi: 10.1016/j.ins.2010.08.045.

[10] Enrique Alba and Marco Tomassini. Parallelism and evolutionary algorithms. *IEEE Transactions on Evolutionary Computation*, 6(5):443–462, 2002. doi: 10.1109/TEVC.2002.800880.

[11] Erick Cantú-Paz. *Efficient and Accurate Parallel Genetic Algorithms*, volume 1 of *Genetic Algorithms and Evolutionary Computation*. Kluwer Academic Publishers, 2001. ISBN 978-0-7923-7221-9. doi: 10.1007/978-1-4615-4369-5.

[12] Calogero Orlando and Angela Ricciardello. Analytic solution of the continuous particle swarm optimization problem. *Optimization Letters*, 15(6):2005–2015, 2021. doi: 10.1007/s11590-020-01671-3.

[13] Hassan M. Emara and Hossam A. Abdel Fattah. Continuous swarm optimization technique with stability analysis. In *Proceedings of the 2004 American Control Conference*, pages 2811–2817, Boston, MA, USA, 2004. doi: 10.23919/ACC.2004.1383892.

[14] Maurice Clerc and James Kennedy. The particle swarm – explosion, stability, and convergence in a multidimensional complex space. *IEEE Transactions on Evolutionary Computation*, 6(1):58–73, 2002. doi: 10.1109/4235.985692.

[15] Hui Huang, Jinniao Qiu, and Konstantin Riedl. On the global convergence of particle swarm optimization methods. *Applied Mathematics & Optimization*, 88(2): Paper No. 30, 2023. doi: 10.1007/s00245-023-09983-3.

[16] Mitchell A. Potter and Kenneth A. De Jong. A cooperative coevolutionary approach to function optimization. In Yuval Davidor, Hans-Paul Schwefel, and Reinhard Männer, editors, *Parallel Problem Solving from Nature – PPSN III*, volume 866 of *Lecture Notes in Computer Science*, pages 249–257. Springer, 1994. doi: 10.1007/3-540-58484-6_269.

[17] James Kennedy and Rui Mendes. Population structure and particle swarm performance. In *Proceedings of the 2002 Congress on Evolutionary Computation (CEC'02)*, volume 2, pages 1671–1676, Honolulu, HI, USA, 2002. IEEE. doi: 10.1109/CEC.2002.1004493.

[18] Nikolaus Hansen, Anne Auger, Raymond Ros, Olaf Mersmann, Tea Tušar, and Dimo Brockhoff. COCO: a platform for comparing continuous optimizers in a black-box setting. *Optimization Methods and Software*, 36(1): 114–144, 2021. doi: 10.1080/10556788.2020.1808977.

[19] Konstantinos Varelas, Ouassim Ait Elhara, Dimo Brockhoff, Nikolaus Hansen, Duc Manh Nguyen, Tea Tušar, and Anne Auger. Benchmarking large-scale continuous optimizers: the bbob-largescale testbed, a COCO software guide and beyond. *Applied Soft Computing*, 97: 106737, 2020. doi: 10.1016/j.asoc.2020.106737.

[20] Daniel Bratton and James Kennedy. Defining a standard for particle swarm optimization. In *Proceedings of the 2007 IEEE Swarm Intelligence Symposium*, pages 120–127. IEEE, 2007. doi: 10.1109/SIS.2007.368035.

[21] Ran Cheng and Yaochu Jin. A competitive swarm optimizer for large scale optimization. *IEEE Transactions on Cybernetics*, 45(2):191–204, 2015. doi: 10.1109/TCYB.2014.2322602.